

Cleaning bronze into silver

Handling nulls, standardising types, ANSI mode, malformed records, and writing silver so it can be rebuilt safely.

Cleaning is not tidying. It is **deciding what a bad value means** and encoding that decision, so nobody downstream has to guess.

The bronze to silver pass

01

Read bronze

As it arrived.
Nothing was
cleaned on the
way in.

02

Standardise types

Strings become
dates, decimals,
integers.

03

Handle nulls

Drop, fill, or keep.
Each is a
decision.

04

Normalise values

Trim, case,
codes, units.

05

Deduplicate and write

One row per
real-world thing,
into silver.

WHY THE ORDER MATTERS

Every step is repeatable because bronze still holds the original. That is the whole reason bronze is append-only.

From **Databricks Runtime 17.0**, ANSI mode is on by default. A failed cast now **throws an error** rather than returning null.

The same cast, two behaviours

EXPRESSION	ANSI ON (DEFAULT FROM DBR 17)	ANSI OFF (LEGACY)
<code>CAST('a' AS INT)</code>	CAST_INVALID_INPUT error	null
<code>CAST(2147483648L AS INT)</code>	CAST_OVERFLOW error	-2147483648, wrapped
<code>try_cast('a' AS INT)</code>	null	null

ON THE EXAM

`try_cast` behaves identically in both modes. That is what makes it the safe choice when null is genuinely the meaning you want.

How to think about the change

- ◆ Each new error is a row that was previously being corrupted silently. The failure is information.
- ◆ Disabling ANSI mode to make errors go away restores the silent corruption. It is the wrong instinct.
- ◆ Switch to `try_cast` where null is the correct meaning, per expression rather than globally.
- ◆ Widen the type where overflow is the issue, for example INT to BIGINT.

ON THE EXAM

`spark.sql.storeAssignmentPolicy` also defaults to ANSI, and is independent. It rejects invalid casts during table insertion.

A null is not zero and not an empty string. It means **unknown**, and Spark's functions treat it that way consistently.

Working with nulls

OPERATION	WHAT IT DOES
<code>isNull()</code> / <code>isNotNull()</code>	Test for null. Never use <code>= NULL</code> , which is always null.
<code>df.na.drop()</code> / <code>dropna()</code>	Remove rows containing nulls
<code>df.na.fill(0)</code> / <code>fillna(0)</code>	Replace nulls with a value
<code>coalesce(a, b, c)</code>	First non-null of several columns
<code>nullif(a, b)</code>	Return null when two values are equal
<code>ifnull(a, b)</code> / <code>nvl(a, b)</code>	Return b when a is null

ON THE EXAM

Equality never matches null. `WHERE col = NULL` returns no rows, always, and it is a silent bug rather than an error.

dropna is more configurable than people expect

- ◆ **how='any'** drops a row if any column is null. This is the default.
- ◆ **how='all'** drops a row only if every column is null.
- ◆ **subset=['a','b']** considers only those columns, ignoring nulls elsewhere.
- ◆ **thresh=3** keeps rows with at least that many non-null values, and overrides how.

ON THE EXAM

The default drops a row if any single column is null. On a wide table with optional fields, that can discard almost everything.

How nulls change your numbers

- ◆ **Aggregates skip nulls.** avg of 10, 20 and null is 15, not 10.
- ◆ **count(*) counts rows. count(col) counts non-null values.** They differ whenever nulls exist.
- ◆ **An inner join drops rows with a null key,** because null never equals null.
- ◆ **So filling nulls with zero changes averages,** while leaving them changes counts. Neither is automatically right.

ON THE EXAM

This is the most commonly missed behaviour in the whole domain. Decide what the null means before choosing.

Standardising types

- ◆ **Dates:** `to_date` and `to_timestamp` with an explicit format string. Never rely on the default parse.
- ◆ **Money:** use **DECIMAL with an explicit precision and scale**, not **DOUBLE**. Floating point loses pennies.
- ◆ **Booleans arriving as text** may be Y, yes, true, 1 or T. Map them explicitly.
- ◆ **Watch for placeholder values:** 1900-01-01, -1, 'N/A', 9999. Convert these to null with `nullif`.

ON THE EXAM

A cast that succeeds is not a cast that is correct. A date parsed with the wrong format can produce a valid but wrong date.

Standardising strings

- ◆ **trim** removes leading and trailing whitespace, which otherwise breaks joins and grouping.
- ◆ **upper or lower** makes case consistent, so 'UK' and 'uk' stop being two countries.
- ◆ **regex_replace** strips or rewrites patterns, such as removing punctuation from phone numbers.
- ◆ **An empty string is not null.** Convert it deliberately if that is what you mean.

ON THE EXAM

Trailing whitespace is invisible in output and fatal to a join. It is worth trimming key columns as a matter of habit.

Malformed records at read time

MODE	BEHAVIOUR
PERMISSIVE	Sets unparseable fields to null and keeps the row. The default.
DROPMALFORMED	Silently discards rows that cannot be parsed
FAILFAST	Throws an exception on the first malformed record

ON THE EXAM

With PERMISSIVE, set `columnNameOfCorruptRecord` to capture the raw text of bad rows rather than losing it. `badRecordsPath` writes bad records to files.

How should silver be written?

OVERWRITE

Rebuild the whole table

- Simple, and always consistent with bronze
- Re-runs are naturally idempotent
- Cost grows with total history, not new data
- Fine while the table is small

MERGE INTO

Apply changes

- Updates matched rows, inserts new ones
- Cost tracks what changed
- Handles late-arriving corrections properly
- Needs a reliable key to match on

THE TIEBREAKER

Append is the third option, and the dangerous one. It is only safe if you can guarantee the same row never arrives twice.

Making silver rebuildable

- ◆ **Aim for idempotency:** running the job twice should leave the table in the same state as running it once.
- ◆ **MERGE on a business key is idempotent.** The second run matches and updates rather than duplicating.
- ◆ **Append is not idempotent** unless something upstream guarantees each row arrives once.
- ◆ **Carry ingestion metadata through:** source file, load timestamp. It makes a bad batch traceable.

ON THE EXAM

The test to apply: if this job runs twice by accident, does anything break? If yes, the design is fragile.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Casts started failing after a runtime upgrade	ANSI mode, on by default from DBR 17
A bad value should become null, not an error	try_cast, which behaves the same in both modes
The average changed after filling nulls	Aggregates skip nulls; zeros participate
count(*) and count(col) disagree	That column contains nulls
Rows vanished after an inner join	Null keys never match, not even each other
Almost every row was dropped by dropna	Default is how='any'. Use subset.
Malformed rows became rows of nulls	PERMISSIVE mode, the read default
The job must be safe to re-run	MERGE on a key, or overwrite. Not append.

ON THE EXAM

Eight lines. The first is the newest and the third is the most commonly missed.

A silver pipeline has run nightly for two years without incident. It casts a quantity column from string to integer. After the platform team upgrades the cluster from Databricks Runtime 15.4 LTS to 17.3 LTS, the job fails with `CAST_INVALID_INPUT`.

An engineer proposes setting `spark.sql.ansi.enabled` to false so the job runs again tonight.

What should the team do?

- A Set `spark.sql.ansi.enabled` to false; the pipeline worked for two years and the data has not changed
- B Recognise that these rows were previously becoming silent nulls, and decide deliberately: `try_cast` where null is correct, or reject the rows
- C Roll back to Databricks Runtime 15.4 LTS permanently
- D Change the column to `DOUBLE`, which accepts a wider range of inputs

◆ PAUSE AND DECIDE

What should the team do?

A Disable ANSI mode

Makes the error go away and restores the silent corruption underneath it. Defensible as a temporary unblock, wrong as the answer.

C Roll back the runtime permanently

Freezes the platform to preserve a bug, and forfeits every later improvement and security fix.

B Treat the errors as information and decide per expression

Correct. The rows were always bad; ANSI mode simply revealed them. `try_cast` where null is the right meaning, rejection where it is not.

D Use DOUBLE instead

Misreads the error. `CAST_INVALID_INPUT` means the text is not a number at all, so a wider numeric type parses it no better.

THE SIGNAL

'Ran for two years without incident' is the false comfort. It ran for two years turning bad values into nulls without telling anyone.

A silver table has a `delivery_rating` column, populated only when a customer chose to leave one. About 40% of rows have a null there. A dashboard reports the average rating as 4.2.

An engineer, tidying the pipeline, adds `fillna(0)` across all numeric columns 'to avoid nulls downstream'. The dashboard now reports 2.5, and nobody can explain the drop.

What happened?

- ☐ A `fillna` corrupted the ratings column by overwriting real values
- ☐ B Averages skip nulls, so the 40% were previously excluded; filling them with 0 made them count as zero ratings
- ☐ C The dashboard is caching the old value and needs refreshing
- ☐ D `fillna` changed the column type, causing an incorrect calculation

◆ PAUSE AND DECIDE

What happened?

- A

fillna overwrote real values

fillna only replaces nulls. Existing ratings are untouched, which is why the answer lies in what the nulls became.
- B

Averages skip nulls; zeros now participate

Correct. 40% of rows moved from being excluded from the average to being counted as ratings of zero, which drags the mean down.
- C

The dashboard is caching

A cache would show the old number, not a new and lower one. The value changed because the data changed.
- D

fillna changed the column type

Filling an integer column with 0 does not change its type, and a type change would not produce a plausible but wrong average.

THE SIGNAL

A null rating means 'did not rate'. A zero means 'rated it zero'. The engineer changed the meaning of 40% of the data.

A bronze customer table has 40 columns. Twelve are genuinely optional: a second address line, a middle name, a mobile number and similar. Two are essential: `customer_id` and `email`. A cleaning step calls `df.dropna()` with no arguments. The silver table ends up with 6% of the expected rows.

What should the call be instead?

- ☐ A `df.dropna(how='all')`
- ☐ B `df.dropna(subset=['customer_id', 'email'])`
- ☐ C `df.dropna(thresh=40)`
- ☐ D `df.fillna("")` followed by `df.dropna()`

◆ PAUSE AND DECIDE

What should the call be instead?

- A

`how='all'`
Only drops rows where every one of the 40 columns is null. That is almost no rows, so bad records with a missing customer_id survive.
- C

`thresh=40`
Requires all 40 columns to be non-null, which is stricter than the default and would keep even fewer rows.
- B

`subset=['customer_id', 'email']`
Correct. It drops rows missing something essential and ignores nulls in the twelve genuinely optional columns.
- D

`fillna("") then dropna()`
Keeps every row by making optional nulls into empty strings, and also disguises missing customer_ids as empty strings.

THE SIGNAL

The default is `how='any'` across all 40 columns. With twelve optional fields, almost every real customer has at least one null.

A silver table is built by reading the last 24 hours of bronze, cleaning it, and appending to silver. Late one evening a job fails halfway and is retried by the on-call engineer. It succeeds. The next day, revenue figures are roughly 40% too high for that date. The bronze table is correct.

What is the design problem?

- ☐ A The retry duplicated rows because appending is not idempotent
- ☐ B The cleaning logic double-counted currency conversions
- ☐ C Bronze contained duplicates that the cleaning step failed to remove
- ☐ D The failed run left an uncommitted transaction that was counted twice

◆ PAUSE AND DECIDE

What is the design problem?

- A

 Append is not idempotent, so the retry duplicated rows
Correct. The first run appended part of the window before failing; the retry appended the whole window again. The overlap is duplicated.
- B

 Currency conversion double-counted
Would affect every day, not just the one that was retried. The symptom is tied to the retry.
- C

 Bronze contained duplicates
The scenario states bronze is correct. Do not argue with a fact the question supplies.
- D

 An uncommitted transaction was counted twice
Delta writes are atomic. An incomplete write commits nothing, so there is no partial transaction to count.

THE SIGNAL

Delta's atomicity means the failed run's committed batches survive. Atomic per write is not the same as atomic per job.

Every cleaning step changes a meaning.
Filling a null, dropping a row and casting a value are all decisions, and the exam tests whether you know which one you made.